GUIDE 36 · OPERATE

Read the logs and trace one request end to end

The latest version is at anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?g=36-trace-a-request-in-the-logs. This copy was made on 4 November 2026.

Time: 10 minutes. **You'll end up with:** a way to go from "a partner says their call failed" - or a red run in the Console - to the exact log lines behind it, using the request ID that ties them together.

Every request has an ID

Every response carries `X-Request-Id`, and every log line written while handling the request carries the same ID and the name of the key that made it:

```shell
curl -i 'http://127.0.0.1:5000/q/orders_by_status?status=paid' -H 'X-API-Key: <app-alice secret>'
# X-Request-Id: 4e3ddfd2d7fc
```

```text
2026-10-04 19:20:09,463 INFO queryapigate [4e3ddfd2d7fc key=app-alice]: Executing on shop (sqlite),
limit=10 offset=0 timeout=30.0: SELECT id, total, status FROM orders WHERE status = :status ORDER BY id
2026-10-04 19:20:09,464 INFO queryapigate [4e3ddfd2d7fc key=app-alice]: GET /q/orders_by_status → 200
in 1.3ms
```

It's in three more places:

- **Error bodies** - `{"error": "...", "code": "...", "request_id": "4e3ddfd2d7fc"}`. A caller who reports an error can send you that.
- **Run history** - each run's `request_id`, in the query's **History** tab and `GET /api/v1/history`.
- **Live events** - in each event's `entry`.

Bring your own ID

A caller can send its own `X-Request-Id` - 1 to 64 letters, digits, `.`, `_`, `:` or `-` (a UUID fits) - and it's used everywhere instead. That ties a run to a trace or ticket in the caller's own system. Verified:

```shell
curl -i 'http://127.0.0.1:5000/q/orders_by_status?status=paid' -H 'X-API-Key: ...' -H 'X-Request-Id:
checkout-7f3a9c'
# X-Request-Id: checkout-7f3a9c       (and the log lines and the history entry say checkout-7f3a9c)
```

Anything else (`has spaces!`) is ignored, not rejected: the response gets a generated ID instead. Always read the ID from the response header, not from what you sent. It's for correlation only - nothing is authorized by it, and two requests can share one if a caller reuses it.

Trace a failure

A scoped key's call to a saved query fails. The caller sees only a generic message - the database's own error can reveal table and column names to someone who didn't write the SQL:

```json
{"error": "An error occurred while executing the SQL query", "code": "query_failed", "request_id": "ticket-5150"}
```

(The admin key gets the same body plus `"detail": "no such column: totl"`.)

Search the log for the ID:

```shell
grep -A 12 'ticket-5150' /var/log/queryapigate.log      # or: docker compose logs queryapigate | grep -A 12 ...
```

```text
INFO queryapigate [ticket-5150 key=reporter]: Executing on shop (sqlite), limit=10 offset=0 timeout=30.0: SELECT id, totl FROM orders
ERROR queryapigate [ticket-5150 key=reporter]: Query on shop failed
Traceback (most recent call last):
  ...
sqlite3.OperationalError: no such column: totl
INFO queryapigate [ticket-5150 key=reporter]: GET /q/order_report → 500 in 3.0ms
```

Who (`reporter`), what (the SQL), why (`no such column: totl`) and how it ended (`500`). Use `-A`: the traceback's own lines don't repeat the ID.

From the Console instead: open the query's **History** tab, find the red run, and take its `request_id` to the log.

These lines need `QUERYAPIGATE_LOG_REQUESTS` left on, as it is by default. With it `off`, a request leaves only its warnings and errors in the log, still tagged with the ID - and the run history keeps the `request_id` either way.

JSON logs, for a log system

```shell
QUERYAPIGATE_JSON_LOGS=1 queryapigate serve
```

One JSON object per line, with the ID and key as fields - and structured fields on the lines you'll want to filter or chart:

```json
{"time": "2026-10-04T19:20:10", "level": "INFO", "logger": "queryapigate", "request_id": "slow-one", "key": "admin",
 "message": "GET /q/example_all_rentals → 200 in 19.0ms",
 "method": "GET", "path": "/q/example_all_rentals", "status": 200, "duration_ms": 19.0, "serialization_ms": 2.7}
```

LINE	EXTRA FIELDS
each request	`method`, `path`, `status`, `duration_ms`, `serialization_ms` (paged formats)
each query	`connection`, `dialect`, `limit`, `offset`, `timeout`, `sql_hash`
a slow query	`connection`, `dialect`, `duration_ms`

A traceback goes into an `exception` field of the same object, so a failure stays one line. `sql_hash` (SHA-256 of the SQL) lets you count or group statements without searching the text.

Slow queries

A query taking at least `QUERYAPIGATE_SLOW_QUERY_THRESHOLD` seconds (default `1`; `0` turns it off) is logged as a warning:

```text
WARNING queryapigate [slow-one key=admin]: Slow query on examples (sqlite): 9.8ms - SELECT r.rental_id,
...
```

(That's with the threshold set to 1 ms, to see it.) A query that's *usually* slow also raises a `query_slow` alert in the Console, and shows in the dashboard's per-connection latency - see Wire up Prometheus and Grafana.

What's never logged

API key secrets, JWTs and connection passwords. The SQL is logged in full - including literal values someone typed into ad-hoc SQL - so treat the log as you would the database's own query log. Bound parameter *values* are not logged in the query line.

Next steps

- Wire up Prometheus and Grafana.
- Read the audit log - configuration changes, which the request log doesn't track.